🟩 WP 5.3 – Requirements with Category, Priority & Rationale

🔷 1. Functional Requirements (FR)

🟢 FR1 – Data Preparation & Feature Engineering

Category: Functional
Priority: 🔴 Critical

Requirement:
System shall ingest processed data from WP 5.1 and perform data alignment, cleaning, and feature engineering to create model-ready datasets.

Rationale:
Raw KPIs are insufficient for modeling. Feature engineering captures temporal behavior, interactions, and context necessary for accurate system identification.

🟢 FR2 – System Identification Model Development

Category: Functional
Priority: 🔴 Critical

Requirement:
System shall generate models mapping process KPIs to quality outputs using system identification and/or data-driven methods.

Rationale:
This model is the foundation for prediction, analysis, optimization, and digital twin functionality.

🟢 FR3 – Model Validation (Pre-deployment)

Category: Functional
Priority: 🔴 Critical

Requirement:
System shall validate models using independent datasets and compute performance metrics before deployment.

Rationale:
Ensures model reliability and prevents propagation of incorrect predictions into downstream decisions.

🟢 FR4 – Behavioral Digital Twin (Simulation)

Category: Functional
Priority: 🔴 Critical

Requirement:
System shall simulate process behavior and generate predicted outputs using the identified model.

Rationale:
Enables prediction without executing on the real system, reducing risk and enabling proactive decisions.

🟢 FR5 – Scenario Simulation

Category: Functional
Priority: 🟠 High

Requirement:
System shall support what-if simulations by varying process parameters and conditions.

Rationale:
Allows safe evaluation of alternatives and supports engineering decision-making.

🟢 FR6 – KPI Correlation & Sensitivity Analysis

Category: Functional
Priority: 🟠 High

Requirement:
System shall compute KPI correlations, feature importance, and sensitivity.

Rationale:
Identifies key drivers of defects and enables targeted optimization.

🟢 FR7 – Root Cause Analysis

Category: Functional
Priority: 🟠 High

Requirement:
System shall identify root causes of defects using model-based and data-driven analysis.

Rationale:
Essential for eliminating defects rather than reacting to them.

🟢 FR8 – Optimization Engine

Category: Functional
Priority: 🔴 Critical

Requirement:
System shall compute optimal process parameters under defined objectives and constraints.

Rationale:
Transforms insights into actionable improvements in quality and productivity.

🟢 FR9 – Safe Operating Window

Category: Functional
Priority: 🔴 Critical

Requirement:
System shall determine safe operating ranges for process parameters.

Rationale:
Prevents operation in unstable regions and reduces defect risk.

🟢 FR10 – Virtual Commissioning (VC) Validation

Category: Functional
Priority: 🟠 High

Requirement:
System shall validate optimized parameters and corrected paths in a virtual environment.

Rationale:
Ensures feasibility and safety before real-world execution.

🟢 FR11 – NC Alignment / Static Correction

Category: Functional
Priority: 🔴 Critical

Requirement:
System shall update NC programs based on reference frame corrections.

Rationale:
Ensures alignment between planned and actual geometry for accurate execution.

🟢 FR12 – MES Integration

Category: Functional
Priority: 🔴 Critical

Requirement:
System shall send optimized parameters and corrected programs to MES for execution and traceability.

Rationale:
Ensures optimized decisions are applied in production and tracked.

🟢 FR13 – Runtime Validation (Validation Loop 2)

Category: Functional
Priority: 🔴 Critical

Requirement:
System shall compare predicted outputs with actual outputs during operation.

Rationale:
Ensures the model remains valid under real-world conditions.

🟢 FR14 – Model Performance Monitoring

Category: Functional
Priority: 🔴 Critical

Requirement:
System shall monitor prediction error, confidence, and model performance.

Rationale:
Maintains trust in the model and detects degradation early.

🟢 FR15 – Model Drift Detection

Category: Functional
Priority: 🔴 Critical

Requirement:
System shall detect drift in model accuracy due to system changes.

Rationale:
Real systems evolve; drift detection ensures continued model relevance.

🟢 FR16 – Model Update & Retraining

Category: Functional
Priority: 🔴 Critical

Requirement:
System shall retrain models using new operational data when required.

Rationale:
Enables continuous improvement and long-term accuracy.

🟢 FR17 – Output to WP 5.1 (Predictions)

Category: Functional
Priority: 🔴 Critical

Requirement:
System shall provide predicted outputs to WP 5.1 for visualization.

Rationale:
Supports proactive monitoring and decision-making.

🟢 FR18 – Output to WP 5.1 (Insights & Recommendations)

Category: Functional
Priority: 🟠 High

Requirement:
System shall provide insights, optimization recommendations, and alerts to WP 5.1.

Rationale:
Enables actionable intelligence at the HMI level.

🟢 FR19 – Output to WP 5.1 (Model Health)

Category: Functional
Priority: 🟠 High

Requirement:
System shall provide model confidence and validation status.

Rationale:
Prevents misuse of unreliable predictions.

🔷 2. Operational Requirements (OR)

🟡 OR1 – Offline and Online Modes

Category: Operational
Priority: 🔴 Critical

Requirement:
System shall support both offline (training/analysis) and online (prediction/validation) modes.

Rationale:
Supports full lifecycle from development to deployment.

🟡 OR2 – Continuous Learning Loop

Category: Operational
Priority: 🔴 Critical

Requirement:
System shall support continuous learning through closed-loop feedback.

Rationale:
Ensures system adapts to changing conditions.

🟡 OR3 – Non-disruptive Operation

Category: Operational
Priority: 🔴 Critical

Requirement:
System shall operate without interrupting production processes.

Rationale:
Critical for industrial deployment.

🔷 3. Performance Requirements (PR)

🟡 PR1 – Prediction Latency

Category: Performance
Priority: 🔴 Critical

Requirement:
System shall provide predictions within defined real-time constraints.

Rationale:
Timely predictions are required for decision-making.

🟡 PR2 – Optimization Response Time

Category: Performance
Priority: 🟠 High

Requirement:
System shall compute optimization results within acceptable time.

Rationale:
Delayed optimization reduces operational value.

🟡 PR3 – Data Throughput

Category: Performance
Priority: 🟠 High

Requirement:
System shall handle high-frequency data streams.

Rationale:
Industrial systems generate large volumes of data.

🔷 4. Reliability & Robustness (RR)

🟡 RR1 – Noise Handling

Category: Reliability
Priority: 🔴 Critical

Requirement:
System shall handle noisy and incomplete data robustly.

Rationale:
Industrial environments are inherently noisy.

🟡 RR2 – Fallback Mechanism

Category: Reliability
Priority: 🟠 High

Requirement:
System shall provide fallback behavior when model confidence is low.

Rationale:
Prevents incorrect decisions based on unreliable predictions.

🔷 5. Scalability & Maintainability (SM)

🟡 SM1 – Scalability

Category: Scalability
Priority: 🟠 High

Requirement:
System shall scale with increasing data, KPIs, and machines.

Rationale:
Future expansion is expected.

🟡 SM2 – Maintainability

Category: Maintainability
Priority: 🔴 Critical

Requirement:
System shall support modular updates and model versioning.

Rationale:
Frequent updates are required in learning systems.

🔷 6. Interoperability & Integration (II)

🟡 II1 – Integration with WP 5.1, MES, VC

Category: Interoperability
Priority: 🔴 Critical

Requirement:
System shall integrate with WP 5.1, MES, and VC systems.

Rationale:
Ensures end-to-end functionality.

🔷 7. Traceability & Security (TS)

🟡 TS1 – Traceability

Category: Traceability
Priority: 🟠 High

Requirement:
System shall log model versions, inputs, and decisions.

Rationale:
Supports audit and debugging.

🟡 TS2 – Security

Category: Security
Priority: 🟠 High

Requirement:
System shall ensure secure data exchange.

Rationale:
Protects industrial data integrity.

🔷 Final MBSE insight

👉 Priority logic:

- 🔴 Critical → core system cannot function without it
- 🟠 High → important for effectiveness
- 🟡 Medium → enhances usability/scalability

🔷 Final one-line summary

👉 **Critical requirements ensure model correctness, prediction, optimization, and feedback; high-priority requirements enhance insight and usability; others ensure scalability and robustness.**

If you want next:
✅ I can map these to **SysML diagrams (Requirement → Block → Interface)**
✅ Or convert to **Jira epics + stories (ready for execution)**

can you give slightly more detailed set of requiremnts? please don't use complicated english

Got it 👍 — I'll give you a **slightly more detailed requirement set**, but in **simple, clear English**, so you can use it directly in docs or reviews.

I'll keep:

- clear "shall" statements
- simple explanation (why)
- priority + category

🟩 WP 5.3 – Process Optimization & Digital Twin

🔷 Detailed Requirements (Simple & Clear)

🔷 1. Data Preparation

🟢 FR1 – Data Intake from WP 5.1

Category: Functional | **Priority:** 🔴 Critical

Requirement:
System shall receive processed KPIs, defect data, and execution data from WP 5.1.

Why:
Without this data, no model or prediction is possible.

🟢 FR2 – Data Cleaning

Category: Functional | **Priority:** 🔴 Critical

Requirement:
System shall clean the data by handling missing values, wrong values, and noise.

Why:
Bad data will lead to wrong model and wrong decisions.

🟢 FR3 – Data Alignment

Category: Functional | **Priority:** 🔴 Critical

Requirement:
System shall align data based on time, position, and process step.

Why:
All signals must match correctly (same time / same location) for proper learning.

🟢 FR4 – Feature Engineering

Category: Functional | **Priority:** 🔴 Critical

Requirement:
System shall create new useful features from KPIs such as:

- change over time
- average values
- interaction between KPIs

Why:
Models need patterns, not just raw values.

🟢 FR5 – Dataset Creation

Category: Functional | **Priority:** 🔴 Critical

Requirement:
System shall create training, validation, and test datasets.

Why:
Needed to build and validate the model properly.

🔷 2. System Identification (Model Building)

🟢 FR6 – Model Creation

Category: Functional | **Priority:** 🔴 Critical

Requirement:
System shall build a model that maps KPIs to outputs like defect or quality.

Why:
This is the core of digital twin and prediction.

🟢 FR7 – Model Type Support

Category: Functional | **Priority:** 🟠 High

Requirement:
System shall support different model types (simple, advanced, ML-based).

Why:
Different processes may need different models.

🟢 FR8 – Model Training

Category: Functional | **Priority:** 🔴 Critical

Requirement:
System shall train the model using training data.

Why:
Model must learn from past data.

🟢 FR9 – Model Validation (Step 2)

Category: Functional | **Priority:** 🔴 Critical

Requirement:
System shall validate the model using test data and calculate error.

Why:
We must ensure the model is correct before using it.

🔷 3. Digital Twin (Prediction)

🟢 FR10 – Prediction Generation

Category: Functional | **Priority:** 🔴 Critical

Requirement:
System shall generate predictions such as:

- defect probability
- quality
- TCP deviation

Why:
Helps understand future behavior before execution.

🟢 FR11 – Scenario Simulation

Category: Functional | **Priority:** 🟠 High

Requirement:
System shall allow testing of different parameter combinations.

Why:
Helps try ideas without using the real machine.

🔷 4. Analysis & Insights

🟢 FR12 – KPI Correlation

Category: Functional | **Priority:** 🟠 High

Requirement:
System shall identify which KPIs affect output most.

Why:
Helps understand important parameters.

🟢 FR13 – Root Cause Analysis

Category: Functional | **Priority:** 🟠 High

Requirement:
System shall find reasons for defects.

Why:
Fixing root cause is better than reacting to defects.

🟢 FR14 – Sensitivity Analysis

Category: Functional | **Priority:** 🟠 High

Requirement:
System shall check how output changes when inputs change.

Why:
Helps understand stability and risk.

🔷 5. Optimization

🟢 FR15 – Parameter Optimization

Category: Functional | **Priority:** 🔴 Critical

Requirement:
System shall find best parameter values for good quality.

Why:
Improves process performance.

🟢 FR16 – Safe Operating Range

Category: Functional | **Priority:** 🔴 Critical

Requirement:
System shall define safe ranges for parameters.

Why:
Prevents defects and machine issues.

🟢 FR17 – Recommendation Output

Category: Functional | **Priority:** 🟠 High

Requirement:
System shall provide recommended actions and settings.

Why:
Helps operators and system take decisions.

🔷 6. Virtual Commissioning (VC)

🟢 FR18 – Simulation Validation

Category: Functional | **Priority:** 🟠 High

Requirement:
System shall validate optimized parameters in virtual environment.

Why:
Avoids risk before real execution.

🟢 FR19 – Safety Checks

Category: Functional | **Priority:** 🔴 Critical

Requirement:
System shall check collision, reachability, and limits in VC.

Why:
Ensures safe operation.

🔷 7. NC Alignment

🟢 FR20 – NC Correction

Category: Functional | **Priority:** 🔴 Critical

Requirement:
System shall adjust NC program based on new reference.

Why:
Real world differs from design; correction is needed.

🔷 8. MES Integration

🟢 FR21 – Send to MES

Category: Functional | **Priority:** 🔴 Critical

Requirement:
System shall send optimized parameters and NC programs to MES.

Why:
Ensures execution in production.

🟢 FR22 – Receive Feedback

Category: Functional | **Priority:** 🔴 Critical

Requirement:
System shall receive execution data from MES.

Why:
Needed for validation loop.

🔷 9. Closed-loop Feedback (Validation Loop 2)

🟢 FR23 – Prediction vs Actual

Category: Functional | **Priority:** 🔴 Critical

Requirement:
System shall compare predicted outputs with actual outputs from WP 5.1.

Why:
Checks if model is still correct.

🟢 FR24 – Error Calculation

Category: Functional | **Priority:** 🔴 Critical

Requirement:
System shall calculate prediction error.

Why:
Helps measure model performance.

🟢 FR25 – Model Monitoring

Category: Functional | **Priority:** 🔴 Critical

Requirement:
System shall monitor model performance over time.

Why:
Detects degradation early.

🟢 FR26 – Drift Detection

Category: Functional | **Priority:** 🔴 Critical

Requirement:
System shall detect when model is no longer accurate.

Why:
System changes over time.

🟢 FR27 – Model Retraining

Category: Functional | **Priority:** 🔴 Critical

Requirement:
System shall update model using new data.

Why:
Keeps system accurate.

🔷 10. Output to WP 5.1 (Monitoring)

🟢 FR28 – Predicted Outputs

Category: Functional | **Priority:** 🔴 Critical

Requirement:
System shall send predicted outputs to WP 5.1.

Why:
Supports monitoring and decision-making.

🟢 FR29 – Insights

Category: Functional | **Priority:** 🟠 High

Requirement:
System shall send KPI correlation and root cause insights.

Why:
Helps engineers understand process.

🟢 FR30 – Model Health

Category: Functional | **Priority:** 🟠 High

Requirement:
System shall send model confidence and alerts.

Why:
Users should know if model is reliable.

🔷 11. Operational Requirements

🟡 OR1 – Offline & Online Mode

Priority: 🔴 Critical

System shall support both training mode and real-time mode.
Why: Needed for full lifecycle.

🟡 OR2 – Continuous Learning

Priority: 🔴 Critical

System shall continuously improve using new data.
Why: System changes over time.

🔷 12. Performance Requirements

🟡 PR1 – Fast Prediction

Priority: 🔴 Critical

System shall generate predictions quickly.
Why: Needed for real-time use.

🔷 Final simple summary

👉 **5.3 takes data → builds model → predicts → explains → optimizes → validates → improves continuously**

If you want next:
✅ I can convert this into **Jira epics + tasks**
✅ Or map these to your **diagram blocks (Step 1–10)**
✅ Or help you define **acceptance criteria for each requirement**

I can map these to SysML diagrams (Requirement → Block → Interface) please do this as well

Yes. Below is a **SysML-style mapping** for WP 5.3:

Requirement → Function → Block → Interface

You can use this directly for MBSE modelling in Cameo / Capella / Enterprise Architect style.

---

1. Main System Block

Block: `WP5.3_Process_Optimization_System`

This contains the following sub-blocks:

Sub-block	Purpose
`Data_Preparation_Module`	Prepares model-ready data
`System_Identification_Module`	Builds and validates model
`Behavioral_Digital_Twin_Module`	Generates predictions and simulations

`Analysis_Insights_Module`	Finds correlation, root cause, sensitivity
`Optimization_Module`	Finds best parameters and safe windows
`NC_Correction_Module`	Corrects NC program based on reference frame
`MES_Interface_Module`	Exchanges data with MES
`Closed_Loop_Learning_Module`	Compares prediction vs actual and triggers model update
`WP5.1_HMI_Output_Interface`	Sends predictions/insights to WP 5.1

SysML Mapping Table

Req ID	Requirement	Function	Responsible Block	Main Interface
FR1	Receive processed data from WP 5.1	Import KPI, defect, execution data	`Data_Preparation_Module`	`I_WP51_Data_In`
FR2	Clean data	Handle missing/noisy/wrong values	`Data_Preparation_Module`	`I_Data_Quality`
FR3	Align data	Align by time, position, process step	`Data_Preparation_Module`	`I_Aligned_Data`
FR4	Feature engineering	Create derived KPIs and modeling features	`Data_Preparation_Module`	`I_Model_Features`
FR5	Dataset creation	Create train/validation/test datasets	`Data_Preparation_Module`	`I_Model_Dataset`

FR6	Model creation	Build KPI → output model	System_Identification_Module	I_Model_InputOutput
FR7	Model type support	Support simple/ML/hybrid models	System_Identification_Module	I_Model_Config
FR8	Model training	Train model using dataset	System_Identification_Module	I_Trained_Model
FR9	Model validation	Validate model before use	System_Identification_Module	I_Model_Validation_Report
FR10	Prediction generation	Predict defect, quality, TCP deviation	Behavioral_Digital_Twin_Module	I_Predicted_Outputs
FR11	Scenario simulation	Run what-if cases	Behavioral_Digital_Twin_Module	I_Scenario_Results
FR12	KPI correlation	Find KPI relationships	Analysis_Insights_Module	I_KPI_Correlation
FR13	Root cause analysis	Identify defect drivers	Analysis_Insights_Module	I_Root_Cause_Insights
FR14	Sensitivity analysis	Check impact of input changes	Analysis_Insights_Module	I_Sensitivity_Results
FR15	Parameter optimization	Find best parameter settings	Optimization_Module	I_Optimized_Parameters
FR16	Safe operating range	Define stable parameter limits	Optimization_Module	I_Safe_Operating_Window
FR17	Recommendation output	Generate action recommendations	Optimization_Module	I_Process_Recommendations
FR18	VC validation	Send optimized settings for virtual validation	Optimization_Module / VC_Interface	I_VC_Validation_Request

FR19	Safety checks	Receive VC safety status	VC_Interface	I_VC_Validation_Status
FR20	NC correction	Adjust NC using updated reference	NC_Correction_Module	I_Corrected_NC
FR21	Send to MES	Send parameters / corrected NC to MES	MES_Interface_Module	I_MES_Out
FR22	Receive feedback	Receive execution feedback from MES	MES_Interface_Module	I_MES_Feedback
FR23	Prediction vs actual	Compare predicted with actual outputs	Closed_Loop_Learning_Module	I_Prediction_Error
FR24	Error calculation	Calculate model error	Closed_Loop_Learning_Module	I_Error_Metrics
FR25	Model monitoring	Track model performance over time	Closed_Loop_Learning_Module	I_Model_Health
FR26	Drift detection	Detect model degradation	Closed_Loop_Learning_Module	I_Drift_Alert
FR27	Model retraining	Trigger model update	Closed_Loop_Learning_Module → System_Identification_Module	I_Retrain_Request
FR28	Send predictions to WP 5.1	Provide predicted outputs to HMI	WP5.1_HMI_Output_Interface	I_HMI_Predictions
FR29	Send insights to WP 5.1	Provide correlation/root cause insights	WP5.1_HMI_Output_Interface	I_HMI_Insights

FR30	Send model health to WP 5.1	Provide confidence and alerts	`WP5.1_HMI_Output_Interface`	`I_HMI_Model_Health`

Suggested SysML Diagrams

1. Requirement Diagram

Use packages like this:

```
WP5.3 Requirements
├── Data Preparation Requirements
├── System Identification Requirements
├── Digital Twin Requirements
├── Analysis Requirements
├── Optimization Requirements
├── NC / MES / VC Integration Requirements
└── Closed-loop Learning Requirements
```

Each requirement should satisfy one or more blocks.

Example:

```
FR10 Prediction Generation
  «satisfy» Behavioral_Digital_Twin_Module
```

---

2. Block Definition Diagram

```
WP5.3_Process_Optimization_System
├── Data_Preparation_Module
├── System_Identification_Module
├── Behavioral_Digital_Twin_Module
├── Analysis_Insights_Module
├── Optimization_Module
├── NC_Correction_Module
├── MES_Interface_Module
├── Closed_Loop_Learning_Module
└── WP5.1_HMI_Output_Interface
```

---

3. Internal Block Diagram Signal Flow

WP 5.1 Data
↓ I_WP51_Data_In
Data_Preparation_Module
↓ I_Model_Dataset
System_Identification_Module
↓ I_Trained_Model
Behavioral_Digital_Twin_Module
↓ I_Predicted_Outputs
Analysis_Insights_Module
↓ I_Process_Insights
Optimization_Module
↓ I_Optimized_Parameters
MES_Interface_Module / NC_Correction_Module

Actual outputs from WP 5.1
↓
Closed_Loop_Learning_Module
↓ I_Retrain_Request
System_Identification_Module

---

4. Interface Control Diagram

Important interfaces to define:

Interface	From	To
`I_WP51_Data_In`	WP 5.1	WP 5.3 Data Preparation
`I_Model_Dataset`	Data Preparation	System Identification
`I_Trained_Model`	System Identification	Digital Twin
`I_Predicted_Outputs`	Digital Twin	Analysis / WP 5.1
`I_Process_Insights`	Analysis	Optimization / WP 5.1
`I_Optimized_Parameters`	Optimization	MES / VC / Robot

`I_Prediction_Error`	Closed Loop	System Identification
`I_HMI_Insights`	WP 5.3	WP 5.1 HMI

---

5. Activity Diagram Sequence

Receive processed data from WP 5.1
→ Prepare model-ready dataset
→ Train and validate model
→ Deploy model into behavioral digital twin
→ Generate predictions
→ Analyze correlation and root cause
→ Optimize parameters
→ Send recommendations to WP 5.1 / MES
→ Execute process
→ Receive actual data
→ Compare predicted vs actual
→ Retrain model if needed

Key MBSE Traceability Statement

Use this in your document:

> Each WP 5.3 requirement is traced to a functional block, and each functional block exposes defined interfaces to WP 5.1, MES, VC, and NC correction modules. This ensures clear ownership, verification, and integration responsibility.

can you create sysML diagram for this?

Edit

I've created a **full SysML-style diagram** for your WP 5.3 system (including requirement diagram, block definition, internal flow, interfaces, and activity flow with loops).

How to use this:

- You can directly **import this into your slides / Klaxoon / Confluence**
- It already follows:
  - Requirement → Block → Interface mapping
  - Data flow + validation loops
  - Connection with **WP 5.1, MES, VC**

🔍 How to read the diagram (very important)

1️⃣ Top-left → Requirement Diagram

Shows grouping of:

- Data prep
- System identification
- Digital twin
- Optimization
- Closed-loop learning

👉 This is your **MBSE requirement structure**

2️⃣ Top-right → Block Definition Diagram

Shows system architecture:

- Each **module = block**
- External systems:
  - WP 5.1
  - MES
  - VC

👉 This is your **logical architecture**

3️⃣ Middle → Internal Block Diagram (MOST IMPORTANT)

This shows **real signal flow**:

```
WP 5.1 → Data Prep → System ID → Digital Twin
            ↓
        Analysis → Optimization → NC / MES
            ↓
     Closed Loop Learning (Validation Loop)
```

👉 This is what you explain in reviews

4️⃣ Right → Interface Table

Shows:

- Who sends data
- Who receives
- What data

👉 This is very useful for integration discussions

5️⃣ Bottom → Activity Flow (WITH LOOPS)

Includes:

- Step 2 loop → Model validation (before deployment)
- Step 10 loop → Runtime validation (after execution)

👉 This answers your earlier confusion about:

- Step 2 vs Step 3 vs Step 4
- Validation loops

🔴 Key Things I Included (based on your questions)

✔ Two validation loops
✔ Prediction vs actual comparison
✔ Feedback to retrain model
✔ Optimization → VC → MES connection
✔ Output to WP 5.1 (HMI)
✔ Clear separation of:

- Digital Twin (prediction)
- Analysis (understanding)
- Optimization (decision)

⚠️ One important note (for your review meeting)

If someone asks:

👉 *"Where is system identification used?"*

You answer:

- It builds the model (Step 2)
- That model is used in:
  - Digital Twin (Step 3)
  - Analysis (Step 4)
  - Optimization (Step 5)
- Then validated again in real system (Loop 2)

If you want next (recommended)

I can refine this further into:

✅ Version for customer presentation

- Cleaner
- Less dense
- More storytelling

✅ Version for MBSE tool (SysML compliant)

- Requirement IDs linked to blocks
- «satisfy», «trace» relationships

✅ Version matching your Klaxoon style EXACTLY

- Same colors
- Same box shapes
- Same layout as your WP 5.1 diagram